CS4243 Cheatsheet

AY25/26 - Anselm Long

Digital Images

Image Formation Model

The 2D image function is represented by $f(x, y) = i(x, y)r(x, y)$ where $i(x, y)$ is illumination, $0 \leq i(x, y) \leq 255$ and $r(x, y)$ is reflectance and $0 \leq r(x, y) \leq 1$.
Images have a resolution which is measured in bits $b = M \cdot N \cdot k$, where M and N represent image size and k represents bit depth.

Color Representations

Images are represented as a matrix with RGB channels represented as a (R, G, B) tuple.
Moving from colour to grayscale, we multiple the respective values by $W_R = 0.299, W_G = 0.587, W_B = 0.114$.
Normalized RGB - Divide by total RGB values:

$$(r, g, b) = (\frac{R}{R+G+B}, \frac{G}{R+G+B}, \frac{B}{R+G+B})$$

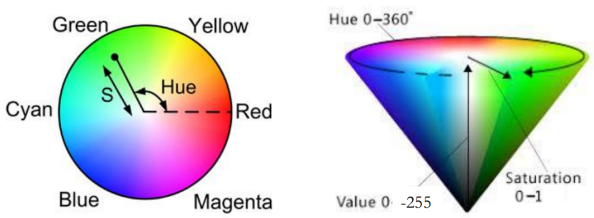
We can represent intensity,

$$I = \frac{R+G+B}{3}$$

.
And as $r + g + b = 1$, we can represent RGB as (r, g, I), and calculate $b = 1 - r - g$.

HSV

Hue - pure colour, 0 - 360
Saturation - purity, mixing pure colour with white light
Value - achromatic mixing from black to white



Point Processing

Transformations

Point Processing - $x_{ij} = f(p_{ij})$
Filtering - $x_{ij} = f(w_{ij})$, where w_{ij} is a window of pixels.

Adjustments

- **Brightness** - $x_{ij} = p_{ij} + b$
- **Contrast** - $x_{ij} = a \cdot p_{ij}$
- **Normalization** - $x_{ij} = \frac{p_{ij} - \mu}{\sigma}$
- **Gamma Mapping** - $x_{ij} = 255 \cdot (\frac{p_{ij}}{255})^{\gamma}$

Histogram

- **Equalization** - forces cumulative distribution to be linear by mapping pixel intensity to a corresponding cumulative value

$$h_k = \sum_{i=1}^{I} \sum_{j=1}^{J} \delta[p_{ij} - k]$$

$$c_k = \frac{\sum_{l=1}^{k} h_l}{IJ}$$

$$x_{ij} = K c_{p_{ij}}$$

- **Stretching** - remaps the min and max of the histogram to fit the entire histogram

$$x = p - f_1 \cdot \frac{255}{f_2 - f_1}$$

- **Thresholding** - Separate foreground and background

Otsu's Method Optimal threshold T* minimizes the sum of weighted variances of object and background.

$$T^* = argmin_T w_1(T) \cdot \sigma_1^2(T) + w_2(T) \cdot \sigma_2^2(T)$$

Correlation Filters

Slide filters over the image. Used for noise reduction etc.
Some filters include:

- **Gaussian Filter** - nearest neighbors have the most effect, σ controls smoothening effect.
- **Sharpening Filter** - accentuates differences between peaks and local average

Template Matching

Matches a section of the image that fits the template given. Uses Normalized Cross Correlation:

$$x_{ij} = \frac{1}{|\mathbf{F}||\mathbf{w}_{ij}|} \sum_{u=-k}^{k} \sum_{v=-k}^{k} f_{uv} \cdot p_{i+u,j+v}$$

$$X = F \otimes P$$

Convolution

Flip kernel in both directions then apply cross correlation. Used for image transformation.

Non-linear Filters

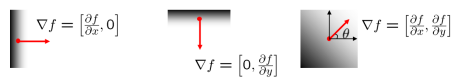
Median Filter - reduce noise and smoothing by removing peaks **Min/Max Filter** - replaces pixel with smallest/largest in the window

Gradients and Edges

Gradient is a vector that points in the direction of most rapid change in intensity.

$$\nabla f = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}\right]$$

The gradient is a vector that points in the direction of most rapid change in intensity.



The **gradient direction** (orientation of edge normal) is given by:

$$\theta = \tan^{-1}\left(\frac{\partial f}{\partial y} / \frac{\partial f}{\partial x}\right)$$

The **edge strength** is given by the gradient magnitude

$$\|\nabla f\| = \sqrt{\left(\frac{\partial f}{\partial x}\right)^2 + \left(\frac{\partial f}{\partial y}\right)^2}$$

Images are discrete. How to take (partial) derivatives of discrete signals?

Take finite differences.

03 Gradients & Edges 8

Direction is given by $\theta = tan^{-1}(\frac{\partial f}{\partial y} / \frac{\partial f}{\partial x})$.

Edge strength is given by gradient magnitude $\sqrt{(\frac{\partial f}{\partial x})^2 + (\frac{\partial f}{\partial y})^2}$

Sobel Filter

Finds edges in images. Horizontal and vertical filters.
Computing Image Gradients
1. Select image gradient filter
2. Convolve with image to obtain components
3. Compute gradient orientation and magnitude

Derivative of Gaussian

As differentiation is sensitive to noise, there is a trade off between noise and edge localization. We can just differentiate h first, then multiply $\frac{\partial}{\partial x}(h * f) = (\frac{\partial}{\partial x} h) * f = (h * g) * f$ Can express the derivation as just another kernel - differentiation is just a convolution operation. Using the Gaussian kernel to blur works but is sensitive to the value of σ.

Laplacian

Laplacian Filter - second-order filter for edge detection. Sensitive to noise.
Laplacian of Gaussian - applying the Gaussian filter first, then Laplacian afterwards

Canny Edge Detector

Non-Maximum Suppression
For each pixel, check if it is a local max along the gradient direction. If so, keep as edge, if not, discard (set to 0). Note that this requires check interpolated values at pixel locations p and r!
Hysteresis Thresholding
Mark pixels that clear high threshold as boundary If there are pixels that clear low threshold and are connected to a boundary, mark them as boundary too.
Steps to do Canny Edge Detector
1. Filter image with derivative of Gaussian
2. Find magnitude and orientation of gradient
3. Non-maximum Suppression on low and high threshold
4. Linking and hysteresis thresholding

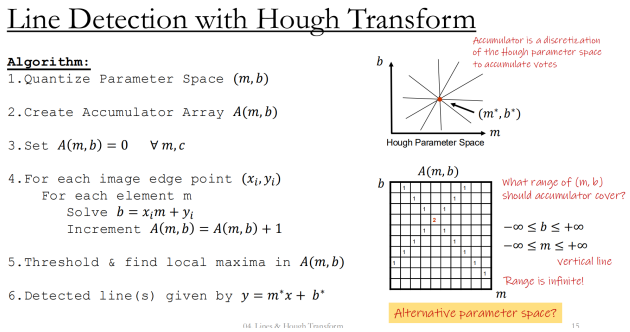
Lines and Hough Transform

Formula for Lines

- **Slope Intercept Form** - $y = mx + b$
- **Double Intercept Form** - $\frac{x}{a} + \frac{y}{b} = 1$
- **Normal Form** - $xcos\theta + ysin\theta = \rho$ (ρ = distance to origin)

Hough Transform

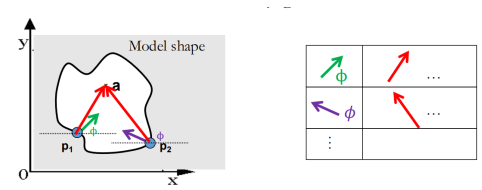
By making the parameters the subject, we can transform the image space to a parameter space.



For the normal form of a line, we can have a point in the image space becoming a sinusoid in the parameter space. Lines on a point become intersecting sinusoids.
Algorithm is the same as the normal form, but the detected line is given by $\rho^* = x \cos\theta^* + y \sin\theta^*$
Line Detection is prone to noise, and might not completely intersect at a point.

Hough Transform for Circles

A point in the image space gets translated to a circle in the parameter space. We swap parameters and variables in the equation.
Offline Modelling Define a model shape by its boundary points p_i and reference point a. At each boundary point, compute a displacement vector, store in a table, indexed by gradient orientation ϕ.



Detection Procedure
For each edge point, use gradient orientation ϕ to index into the stored table, then use the retrieved r vectors to vote for the reference point.

Segmentation

Separate image into coherent regions.

K-means Clustering

1. Randomly initialize cluster centers
2. Determine points in each cluster by distance
   - Assign points $y_i = \arg\min_j ||p_i - c_j||^2$
3. Given the points in each cluster, solve for the centroid, by mean distance - average of all the points
4. If the centroids changed, repeat steps 2 and 3.

Feature Selection for clustering can be based on intensity, color similarity, intensity + position similarity.

SLIC

Algorithm:
1. Compute initial superpixel cluster centers by sampling the image at regular grid steps. Set a label L(p) = -1 and a distance d(p) = ∞
2. Assign samples to cluster centers. For each cluster center, compute the distance between center and each pixel. Then if distance is less than current distance, set distance and set label.
3. Update cluster centers with mean distance, same as k-means
4. Test for convergence. Compute Euclidean norms of differences between mean vectors, compute residual error as sum of norms. If error is less than a threshold, proceed.
5. Replace superpixels with average value to get stained glass effect if necessary.

Mean-Shift Clustering

Find modes or local density maxima in the feature space
For each point, define a window around it to compute the centroid, shift center of the window to that centroid, repeat until stops moving.
Attraction basin: region in feature space for which all trajectories of centroids lead to the same mode
Cluster: all data points in the attraction basin of a mode.

Kernel Density Estimation (KDE)

Assume that data is a sample drawn from some distribution with an unknown density function
KDE estimates the shape of the function depending on the kernel used.

Mean Shift Procedure

1. Compute Mean Shift Vector

$$m_{h,G}(x) = \left[\frac{\sum_{i=1}^{n} x_i g\left(\left\| \frac{\mathbf{x}-\mathbf{x}_i}{h} \right\|^2\right)}{\sum_{i=1}^{n} g\left(\left\| \frac{\mathbf{x}-\mathbf{x}_i}{h} \right\|^2\right)} - \mathbf{x} \right]$$

2. Translate the density window and update: $x' = x + m(x)$, $x = x'$
3. Iterate steps 1 and 2 until convergence

Segmentation with Mean Shift

- Find features to represent each pixel
- Initialize density windows at individual feature points
- Perform mean shift for each point until convergence
- Merge pixels whose feature points end up near the mode or peak into the same segment

Choosing Bandwidth, h - Trial and error or Sample some points, and use an average distance to the k-nearest neighbours. **Speedups** - assume all points within radius of endpoint after mean shift will belong to the same cluster, and we can also assume all points within the radius of the search path from start to mode will be in the cluster.

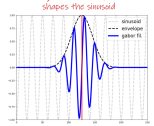
Feature	K-Means Clustering	Mean Shift Clustering
Pros	• simple and fast to compute. • converges to local minimum.	• general mode-finding. • finds variable number of modes. • robust to outliers.
Cons	• hard to set K. • sensitive to initial centers. • detects spherical clusters.	• dependent on bandwidth h. • expensive and slow. • curse of dimensionality.

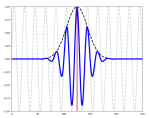
Visual Textures

Filter Banks

A combination of different patterns at various scales and orientations.
Gabor Filters - combining sinusoids with an exponential envelope.

gaussian envelope shapes the sinusoid





$$e^{-\frac{x^2}{2\sigma^2}} \sin(2\pi f x) \qquad e^{-\frac{x^2}{2\sigma^2}} \cos(2\pi f x)$$

frequency of sinusoid

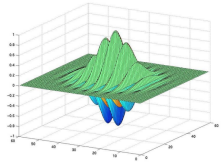
exponential envelope

sinusoid

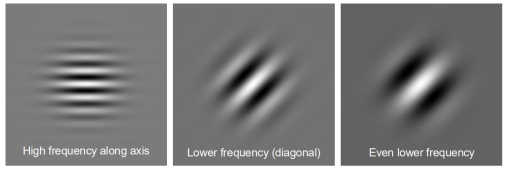
* f corresponds to $\frac{1}{\lambda}$ in next slide

2D Gabor Filters

$$f_{mn} = \frac{1}{2\pi\sigma^2} \exp\left[-\frac{m^2+\gamma^2 n^2}{2\sigma^2}\right] \sin\left[\frac{2\pi(\cos[\omega]m + \sin[\omega]n)}{\lambda} + \phi\right]$$



This is just the real component, used for the odd filter; can also have a version using cosine, for the even filter.



Textons

Characterises texture by replacing pixel with a texture type.
Computing Textons:
1. Apply filter bank to training images
2. Cluster feature space and store cluster centres to form a texton dictionary.
3. For a new test image, filter the image with the same filter bank, and based on feature vectors at each pixel, assign to the nearest cluster.

Histogram of Textons - for a region, compute a histogram of textons as the representation which is a vector storing the number of occurences for each region.
This histogram can then be used for segmentation or clustering.

Perceived Boundaries

Using Texture Gradients (TG) to identify boundaries. Consider a disc, split into two halves by a diameter of a particular orientation. Measure the difference in texture between the two halves by comparing texton histograms. Try all possible orientations
A high distance between histogram suggest a more likely boundary.